Bioinformatics Self-Study Curriculum

A 12-Block, Sprint-Based Program

Statistical Genetics · Multiplexed Assays · Unix · Python · R · AI Tools

Hands-on exercises with real data in every session

Designed for independent learners in genomics, biostatistics, and computational biology.

No instructor required — all resources are freely available or open-access.

Contents

1 How This Curriculum Works (Read This First) 3
1.1 The Three Rules . . . 3
1.2 How a Block Works . . . 3
1.3 Your Toolkit (Set Up in Block 0) . . . 4
1.4 Interleaving: Why This Curriculum Mixes Things Up . . . 4
1.5 How to Use AI as Your Study Partner . . . 4

2 Blocks 1–2: Unix Foundations 6
2.1 Block 1: The Command Line . . . 6
2.2 Block 2: Scripts, Environments & Git . . . 8
2.2.1 Can I move on? Checkpoint: . . . 11
2.2.2 Resources . . . 11

3 Blocks 3–4: Python for Genomic Data 12
3.1 Block 3: Core Python via Biology . . . 12
3.2 Block 4: Pandas, Plotting & Bio Libraries . . . 14
3.2.1 Checkpoint: . . . 15
3.2.2 Resources . . . 15

4 Blocks 5–6: R, Bioconductor & Differential Expression 16
4.1 Block 5: R Basics + Bioconductor . . . 16
4.2 Block 6: Differential Expression + ggplot2 . . . 18
4.2.1 Checkpoint: . . . 19
4.2.2 Resources . . . 19

5 Blocks 7–8: Statistics for Genomics 21
5.1 Block 7: Distributions, Testing & Models . . . 21
5.2 Block 8: Dimensionality Reduction & Bayesian Ideas . . . 22
5.2.1 Checkpoint: . . . 23
5.2.2 Resources . . . 23

6 Blocks 9–10: Applications — Single-Cell & GWAS 24
6.1 Block 9: Single-Cell RNA-seq — Start to Finish . . . 24
6.2 Block 10: GWAS Pipeline — Variant to Function . . . 25
6.2.1 Checkpoint: . . . 26
6.2.2 Resources . . . 26

7 Blocks 11–12: AI, Right Questions & Capstone 27
7.1 Block 11: Asking the Right Questions + AI Mastery . . . 27
7.1.1 The Question Map . . . 27
7.2 Block 12: Reproducible Pipeline + Capstone . . . 28
7.2.1 Capstone Project . . . 28

8 Running Project: From Growth Curves to QTL Mapping 30
8.1 Problem Set 1: Yeast Growth Curves . . . 30
8.1.1 Part A: Generate the Biomek Randomization CSV . . . 30

8.1.2 Part B: Fit Growth Curves & Extract Doubling Times . . . 31
8.1.3 Part C: Visualize & Test for Differences . . . 31
8.1.4 Part D: Permutation Test . . . 31
8.1.5 Part E: Power Analysis by Simulation . . . 31
8.2 Problem Set 2: Heritability & Mixed Models . . . 32
8.2.1 Part A: Read & Merge Data . . . 32
8.2.2 Part B: Visualize . . . 32
8.2.3 Part C: Fixed-Effects Linear Models . . . 33
8.2.4 Part D: Broad-Sense Heritability . . . 33
8.2.5 Part E: Mixed Model with Random Effects . . . 33
8.2.6 Part F: Normalized Response to Chemical X . . . 33
8.2.7 Part G: Heritability of the Normalized Trait . . . 34
8.3 Problem Set 3: Short-Read Alignment & Strain Identification . . . 34
8.3.1 Part A: Identify the Mystery Strain . . . 34
8.3.2 Part B: Visualize Meiotic Crossovers . . . 34
8.4 Problem Set 4: QTL Mapping . . . 35
8.4.1 Part A: Genome-Wide LOD Scan . . . 35
8.4.2 Part B: Permutation Threshold for Significance . . . 35
8.4.3 Part C: 1.5-LOD Drop Confidence Interval . . . 36
8.4.4 Part D: Candidate Genes & Biological Hypothesis . . . 36

9 Appendix A: Tools & Installation **37**
9.1 Core infrastructure (install once, before anything else) . . . 37
9.2 Command-line utilities not always installed by default . . . 37
9.3 Bioinformatics tools (install into a conda env) . . . 38
9.4 R and Python packages . . . 40

10 Appendix B: Programming Concepts Cheat Sheet **41**

11 Appendix C: Reading List (Prioritized) **42**
11.1 Start Here (the essentials) . . . 42
11.2 Go Deeper (when you're ready) . . . 42
11.3 Reference (keep on your shelf) . . . 42
11.4 Online Tools & Databases . . . 42
11.5 12-Block Overview . . . 43

How This Curriculum Works (Read This First)

Traditional curricula front-load reading and back-load doing. This one flips that: every session is hands-on first, theory second.
Every session produces something tangible. You never just "read about" a topic — you DO it, see output, and move on.
It's okay to jump around. Each sprint is self-contained enough that you can follow your curiosity without losing the thread.

The Three Rules

Rule 1: The 25-Minute Sprint. Every study session is broken into 25-minute sprints (Pomodoro). Each sprint has ONE clear goal and ONE tangible output. If you finish early, take the break early. If you're in flow, keep going — but never force yourself past 50 minutes without a break.

Rule 2: Output Before Input. Always DO something before you READ about it. Try the command first. Break something. Then read why it broke. You'll remember mistakes and fixes much better than anything you passively read.

Rule 3: The Checkpoint. Every sprint ends with a concrete thing you made, produced, or proved works. Screenshot it, commit it, or write one sentence in your journal. This is your proof of progress.

How a Block Works

The curriculum is divided into 12 **blocks**. Each block is roughly 2 hours of focused work, organized as a small batch of 25-minute sprints with short breaks between them. Most blocks land around 3–5 sprints — check the chapter for the exact count. How you spread a block across the calendar is up to you: one 2-hour sitting, two 60-minute sittings, or a sprint a day all work. What matters is that you finish the block's sprints before moving on.

Sprint (25 min)	Hands-on: work through the sprint's goal and produce its checkpoint
Break (5 min)	Stand up, walk, look away from the screen
Next sprint	Continue — or stop here and pick up another day
End of block	Journal one sentence per sprint. Commit to Git.

Never force yourself past 50 minutes of sprinting without a real break. If you want to do more in a day, take 30+ minutes off before starting the next block.

Your Toolkit (Set Up in Block 0)

Before Block 1, spend one sitting setting this up. This is Sprint 0. **Appendix A is your complete install reference** — every tool named in this curriculum (core infrastructure, command-line utilities, bioinformatics packages, R and Python libraries) is listed there with macOS and Linux install commands and official documentation links. Keep it open while you work through the curriculum.

Terminal	Everything runs here	Linux/macOS terminal; or WSL2 on Windows — see Appendix A
Miniforge	Package management without pain	`https://github.com/conda-forge/miniforge` — see Appendix A
VS Code	Editor with terminal built in	`https://code.visualstudio.com` (or your favorite editor)
Git + GitHub	Version control & your portfolio	`https://git-scm.com`; make a GitHub account now
Claude or ChatGPT	AI tutor always available	`https://claude.ai` (free tier works)
A journal.md file	One sentence per day = progress proof	Create it in your Git repo right now

Install everything. Open your terminal. Create a Git repo called `bioinfo-journey` with a `journal.md` file. Push it to GitHub.

Interleaving: Why This Curriculum Mixes Things Up

Research consistently shows that retention improves when topics are interleaved (mixed) rather than massed (all-Unix-then-all-Python).
Each block below has a PRIMARY focus, but deliberately connects to other skills. By Block 4 you're already using Python + Unix + statistics together.
If you get stuck on one topic, switch to a different sprint in the same block. Coming back later with fresh eyes is not quitting — it's how interleaving works.

How to Use AI as Your Study Partner

AI tools are effective study aids — treat them as a resource you can query at any time for explanations, debugging, and code review.

- **Stuck on an error?** Paste the FULL error message into Claude. Don't describe it — paste it.
- **Confused by a concept?** Ask: "Explain [concept] like I already know [thing I do know] but not [thing I don't]."
- **Want to check understanding?** Ask: "I think [my understanding]. Is that right? What am I missing?"
- **Need a break from reading?** Ask: "Give me a 5-minute hands-on exercise to practice [concept]."
- **Feeling overwhelmed?** Ask: "What's the ONE most important thing I should understand about [topic]?"

Always run AI-generated code before trusting it. AI hallucinates function names, invents packages, and confuses API versions. Test first, trust after.

Blocks 1–2: Unix Foundations

Primary: Command line, text processing, bash scripting | **Interleave:** Git, project setup

Every bioinformatics pipeline runs on Unix. Genomic data is text. If you can wrangle text in a terminal, you can wrangle genomes.

Every non-default tool named anywhere in this curriculum has install commands (macOS and Linux) and an official link in **Appendix A**. Flip to it the moment you hit a command that isn't on your machine — don't guess.

Block 1: The Command Line

Sprint 1: Navigate & Look Around *(25 min)*

- Open your terminal. Run: `pwd`, `ls`, `ls -la`, `cd ~`, `cd /`, `cd -`, `tree` (install with conda if needed)
- Make a project directory: `mkdir -p ~/bioinfo/block1/{data,scripts,results}`
- Explore: what's in `/usr/bin`? How many files? (`ls /usr/bin | wc -l`)

Screenshot your project directory tree. Paste it in journal.md.

Sprint 2: Pipes — The Unix Superpower *(25 min)*

- The pipe (`|`) sends output from one command into another. This is the entire Unix philosophy.
- Try: `echo 'ATGCGATCGATCG' | grep -o . | sort | uniq -c | sort -rn` — count nucleotide frequencies in one line
- Download a small FASTA file:

```
curl -O https://downloads.yeastgenome.org/sequence/
  S288C_reference/genome_releases/
  S288C_reference_genome_Current_Release.tgz
tar xzf S288C_reference_genome_Current_Release.tgz
```

- Explore it: `grep '>' *.fsa | head` — what are these header lines telling you?

You piped three commands together and saw structured output.

Sprint 3: grep + awk = Data Surgery *(25 min)*

- `grep`: searches for patterns. `grep -c '>' file.fasta` counts sequences. `grep -v` inverts (excludes matches).
- `awk`: extracts and transforms columns. `awk -F'\t' '{print $1, $2, $5}'` pulls chr, pos, and qual from a VCF.
- Grab a small VCF to practice on. Yeast works well — a whole-organism variant file that's only a few MB:

  ```
  curl -O https://ftp.ensembl.org/pub/
    current_variation/vcf/saccharomyces_cerevisiae/
    saccharomyces_cerevisiae.vcf.gz
  gunzip saccharomyces_cerevisiae.vcf.gz
  ```

  If that link has moved, browse `https://ftp.ensembl.org/pub/current_variation/vcf/` and pick any small organism's VCF. Other good "small VCF" sources: the bcftools test data (`github.com/samtools/bcftools`, `test/` directory) or a single-chromosome 1000 Genomes file restricted to a small region with `tabix`.
- Count variants per chromosome with:

  ```
  grep -v '^#' file.vcf | awk '{print $1}' | \
    sort | uniq -c | sort -rn
  ```

You summarized a genome's worth of variants in one line.

Sprint 4: Put It All Together — Pick a Restriction Enzyme for Cloning *(25 min)*

A real scenario: you want to shotgun-clone ∼3 kb fragments of the yeast genome into a plasmid. Three cheap 6-cutters are on the shelf — EcoRI (`GAATTC`), BamHI (`GGATCC`), and HindIII (`AAGCTT`). *Before* ordering anything, predict from the genome sequence how often each enzyme cuts and which one produces fragments closest to your 3 kb target. Do it all with pipes — no scripts yet. Paste every command and its output into `journal.md` as you go.

1. **Confirm and organize.** `head` the yeast FASTA — the first line should start with `>`. Then move the FASTA (and the VCF from Sprint 3) into `data/` in one pipeline: `ls *.fsa *.vcf 2>/dev/null | xargs -I{} mv {} ~/bioinfo/block1/data/`. Verify with `ls ~/bioinfo/block1/data/`.
2. **Measure the genome.** Strip headers and newlines, count characters:

   ```
   grep -v '>' ~/bioinfo/block1/data/*.fsa | \
     tr -d '\n' | wc -c
   ```

   For S288C, expect roughly 12 Mb. Save this number — you'll divide by it next.
3. **Count cut sites per enzyme.** Template for EcoRI:

   ```
   grep -v '>' ~/bioinfo/block1/data/*.fsa | \
     tr -d '\n' | grep -o GAATTC | wc -l
   ```

   Run it three times — once each for `GAATTC`, `GGATCC`, `AAGCTT`. Which cuts most? Fewest?
4. **Why one strand is enough.** All three recognition sites are palindromes (read the same on both strands), so every real cut site is already captured by a forward-strand match. For non-palindromic motifs you'd also have to scan the reverse complement — good to know before you generalize this approach.
5. **Compute average fragment size.** Fragment size ≈ genome length ÷ number of cut sites. Use `bc`: `echo "12157105 / NSITES" | bc` (swap in your actual genome length and cut count). Expect numbers in the range of hundreds of bp to tens of kb.

6. **Pick your enzyme.** Which enzyme's average fragment size is closest to your 3 kb target? Write one sentence in `journal.md` explaining your choice, and flag which enzyme would be a bad idea (cuts so often you'd get tiny shredded fragments, or so rarely the fragments are too big to clone).

Your journal shows the `head` check, the move-into-`data/` pipeline, the genome length, cut-site counts for three enzymes, their average fragment sizes, and your enzyme pick with a one-sentence justification — all produced with pipes.

In Unix bioinformatics, the workflow is almost always: EXTRACT columns (cut/awk) → FILTER rows (grep) → SORT → COUNT (uniq -c). This pattern covers 80% of quick data exploration.

Block 2: Scripts, Environments & Git

Sprint 5: Your First Bash Script *(25 min)*

- Create a file: `nano ~/bioinfo/scripts/count_variants.sh`
- Add: `#!/usr/bin/env bash`, then `set -euo pipefail` (this catches errors immediately)
- Write a script that takes a VCF filename as `$1` and outputs a per-chromosome variant count table
- Make it executable: `chmod +x count_variants.sh`, then run it

You have a reusable script. Run it on a different VCF. Same script, different data.

Sprint 6: Conda & Mamba: Isolated, Reproducible Environments *(25 min)*

- Create an environment: `conda create -n biotools samtools bcftools bedtools fastp`
- Activate it: `conda activate biotools`. Verify: `samtools -version`.
- Export it: `conda env export > environment.yml` — anyone can now recreate your setup with `conda env create -f environment.yml`.
- Deactivate: `conda deactivate`. Run `samtools` again — it's no longer available. Environments isolate dependencies from each other and from the system.
- **Use mamba for speed.** `conda`'s dependency solver is slow on large environments. `mamba` is a drop-in replacement (C++ solver) that resolves the same specs in seconds. Miniforge (https://github.com/conda-forge/miniforge) ships with it; otherwise `conda install -n base -c conda-forge mamba`. Swap it in wherever you'd type `conda`: `mamba create -n biotools samtools bcftools bedtools fastp`.

Conda packages are built per (OS, CPU) pair — `linux-64`, `osx-64`, `osx-arm64` (Apple Silicon), `linux-aarch64` (ARM Linux / some HPCs). Not every bioinformatics package has a build for every platform. On an M1/M2/M3 Mac, a missing `osx-arm64` build usually surfaces as `PackagesNotFoundError`; common fixes are to (1) create the env under Rosetta with `CONDA_SUBDIR=osx-64 conda create ...`, or (2) find an alternative package with native ARM support. An `environment.yml` built on one platform is not guaranteed to solve on another — expect small fix-ups when moving between laptop

and cluster, and note the architecture in a comment at the top of the file.

Your `environment.yml` is a reproducible recipe. Commit it to Git, with a header comment noting which architecture built it.

Sprint 7: Containers — Reproducibility Beyond Conda *(25 min)*

A container is a single image file that bundles an entire Linux userland — OS libraries, the tool you care about, and everything the tool depends on — so it runs bit-for-bit identically on any compatible host. Conda manages packages *inside* ecosystems (Python, R, bioconda); containers freeze the layer *below* that, where system libraries, `glibc` versions, and compiled binaries live. A container is not a virtual machine — there's no guest kernel — so startup is near-instant and overhead is minimal.
Rule of thumb: use conda for interactive development (fast to edit, easy to iterate). Use a container when you ship a pipeline to a cluster, share an analysis with a collaborator on a different OS, or archive a workflow alongside a manuscript — situations where "works on my machine" has to become "works on yours."

- **Two runtimes you'll encounter.** `Docker` is the developer-facing tool — it requires a privileged daemon, so most shared HPCs will not run it. Install Docker Desktop on macOS (https://docs.docker.com/desktop/install/mac-install/) or the Engine on Linux (https://docs.docker.com/engine/install/). `Apptainer` (formerly `Singularity`) is the HPC-safe equivalent: no daemon, no root, and a container is a single `.sif` file you can `scp` around. HPCs usually provide a module (`module avail apptainer`); for a personal Linux install, see https://apptainer.org/docs/user/main/quick_start.html.
- **Pull a biocontainer.** The Biocontainers project auto-builds one container per bioconda package. Try samtools:

  ```
  docker pull \
    quay.io/biocontainers/samtools:1.19--h50ea8bc_0
  ```

- **Run without installing.** Execute the container's `samtools` with no other setup:

  ```
  docker run --rm \
    quay.io/biocontainers/samtools:1.19--h50ea8bc_0 \
    samtools --version
  ```

  `-rm` discards the stopped container after it exits; the image stays cached for next time.
- **Mount your data.** Containers see an isolated filesystem by default. Map your current directory in so the tool can read local files:

  ```
  docker run --rm -v "$PWD":/work -w /work \
    quay.io/biocontainers/samtools:1.19--h50ea8bc_0 \
    samtools view file.bam | head
  ```

  `-v HOST:CONTAINER` mounts a host directory; `-w` sets the working directory inside.
- **On an HPC, use Apptainer.** Same image, no daemon required:

  ```
  apptainer pull \
    docker://quay.io/biocontainers/samtools:1.19--h50ea8bc_0
  apptainer exec samtools_1.19--h50ea8bc_0.sif \
    samtools --version
  ```

Apptainer auto-mounts your home directory and the current working directory, so most pipelines just work.

Conda gives you speed and easy iteration at the cost of reproducibility guarantees — two users with the same `environment.yml` may still resolve to different package builds on different platforms. A container gives you a fixed binary that will run identically for years, at the cost of being heavier to build and harder to edit. Many published pipelines use both: conda during development, a container (often built *from* the conda env) for the frozen release.

You pulled a container, ran a bioinformatics tool from it without installing anything locally, and mounted a host directory into it. You know which runtime (Docker vs. Apptainer) you'd reach for on a laptop vs. an HPC, and why.

Sprint 8: Git + GitHub — Your Time Machine and Portfolio *(25 min)*

- **Initialize and commit.**

```
cd ~/bioinfo && git init -b main && \
  git add -A && \
  git commit -m 'Blocks 1-2: Unix foundations'
```

- Edit something. `git diff` to inspect, `git add`/`git commit` to save, `git log -oneline` to review.
- **.gitignore.** Exclude anything big or generated: `*.bam`, `*.fastq.gz`, `*.vcf.gz`, `*.sam`. Never commit raw data — commit the code that produces it and a pointer to where it lives.
- **Set up an SSH key for GitHub** (one-time; after this, no password prompts):

```
ssh-keygen -t ed25519 -C "you@example.com"
# press Enter to accept the default path;
# optionally set a passphrase
cat ~/.ssh/id_ed25519.pub
```

  Copy the printed public key, then go to github.com → Settings → SSH and GPG keys → New SSH key and paste it. Test: `ssh -T git@github.com` should greet you by username.
- **Publish your repo.** Create an empty repo on github.com (don't add a README — your local copy already has files). Then from your local repo:

```
git remote add origin \
  git@github.com:<you>/bioinfo-journey.git
git push -u origin main
```

  The `-u` flag wires `main` to the remote so future `git push`es need no arguments.
- **Clone and pull.** To download a repo onto a new machine or grab someone else's work: `git clone git@github.com:user/repo.git`. Inside an existing clone, `git pull` fetches new commits from the remote and merges them into your branch — this is how you keep a checkout in sync. Practice: clone your own repo into `/tmp/`, push a change from your main checkout, then run `git pull` in `/tmp/` and confirm the change arrived.

Your bioinfo-journey repo has real commits, ignores big files, pushes over SSH without password prompts, and you've proved you can `clone` and `pull` from it. This is your portfolio.

Sprint 9: Integration Sprint: Full Pipeline *(50 min)*

- Combine everything: write a bash script that creates a conda env, downloads a FASTQ from SRA (use `fastq-dump` or `fasterq-dump` from the SRA Toolkit — `mamba install -c bioconda sra-tools`), runs `fastp` for QC, and logs results to a file.
- Use `tmux` (macOS: `brew install tmux`; Linux: `sudo apt install tmux`) or `screen` so you can disconnect and reconnect (crucial for HPC).
- Commit the script and the QC results (not the raw data) to Git.

A complete, reproducible, version-controlled QC pipeline. From nothing to this in two blocks.

End of Blocks 1–2

Take a day to step away before starting Python. The command-line fundamentals are now in place.

Can I move on? Checkpoint:

- You can navigate any directory, find files, and chain 3+ commands with pipes
- You can write a bash script that processes a genomic file and logs output
- You have a conda environment with bioinformatics tools installed
- Your work is in a Git repo on GitHub

If not, spend 2–3 more days on the gaps. No rush.

Resources

- Software Carpentry: The Unix Shell (free, interactive)
- Buffalo – *Bioinformatics Data Skills*, Ch. 3, 7, 12
- explainshell.com — paste any command and see what each part does

Blocks 3–4: Python for Genomic Data

Primary: Data structures, Pandas, visualization | **Interleave:** Unix pipes, real genomic files

You just learned to process text files with grep and awk. Python does the same things but with more power, reusable code, and the ability to make plots and run statistics.

Block 3: Core Python via Biology

Sprint 1: Data Structures Are Biology *(25 min)*

Pythonic code is less about clever tricks and more about reaching for the right built-in collection. Each has a natural biological use case.

- **Strings = DNA sequences.**

```
seq = 'ATGCGATCG'
comp = seq.translate(str.maketrans('ATCG','TAGC'))
print(comp[::-1]) # reverse complement
```

- **Lists = ordered, mutable collections.** Comprehensions are how you filter them: `[g for g in genes if g.startswith('BRCA')]`. Swap the brackets for parentheses to get a *generator expression* that yields items one at a time without building the full list in memory.
- **Tuples = immutable records.** Genomic coordinates are a natural fit: `peak = ("chr1", 1000, 2000)`. Because tuples are immutable, they're hashable — so they can serve as dict keys or live in sets ("which regions have I already processed?").
- **Dictionaries = lookup tables.** Build a codon table: `codons = {'ATG':'M', 'TAA':'*', 'GCT':'A', ...}` and translate three bases at a time.
- **Sets = membership and overlap.** `set(sampleA) & set(sampleB)` (intersection), `|` (union), `-` (difference). Use them when order doesn't matter and you care about presence/absence.
- **`collections.Counter` and `defaultdict`.** `Counter(seq).most_common(4)` counts every character and returns the top four — no "does this key exist yet?" dance. `defaultdict(list)` lets you write `d[chrom].append(pos)` without initializing each chromosome's list by hand.

You reverse-complemented DNA with `str.translate`, counted nucleotide frequencies with `Counter`, and grouped positions by chromosome with `defaultdict`. Idiomatic Python is about picking the right container, not writing loops.

Sprint 2: Parsing Genomic Files *(25 min)*

- Write a FASTA parser: open the file, loop through lines, yield (header, sequence) pairs
- Use a context manager (`with open(...) as f:`) — always. This prevents resource leaks.
- Parse a BED file into a list of dictionaries: `[{chr: ..., start: ..., end: ...}, ...]`
- Handle errors: what happens when a line is malformed? Add `try/except`.

You can parse any text-based genomic file. This is a skill you'll use indefinitely.

Sprint 3: Generators — Process Files Bigger Than Your RAM *(25 min)*

- A generator uses `yield` instead of `return` — it produces items one at a time without loading everything into memory
- Convert your FASTA parser to a generator. Notice: it handles a 10GB file the same way as a 10KB file.
- This is critical for bioinformatics: VCFs can be billions of lines. You cannot load them into memory.

Your FASTA parser works on any size file. That's production-quality code.

Sprint 4: Pythonic Idioms & Type Hints *(25 min)*

Writing Python that other people (and future-you) can read quickly is less about clever tricks and more about using the idioms the language was built around. Each of these takes five minutes to learn and saves hours of debugging.

- **f-strings.** `f"{gene} on chr{chrom} (padj={pval:.2e})"` — variables, formatting, and expressions in one readable line. Use these exclusively for user-facing strings; they're faster than `%` or `.format()` and much harder to misread.
- **`pathlib` for filesystem paths.**

  ```
  from pathlib import Path
  data = Path.home() / "bioinfo" / "block1" / "data"
  for vcf in data.glob("*.vcf"):
      print(vcf.stem, vcf.stat().st_size)
  ```

  Path objects beat string concatenation: they handle OS differences, support `/` for joining, and expose `.stem`, `.suffix`, `.parent`, `.exists()`.
- **`enumerate` and `zip`.** Almost never write `for i in range(len(xs))` — use `for i, x in enumerate(xs)` or `for a, b in zip(xs, ys)`. These are the canonical ways to walk sequences.
- **Dataclasses for records.**

  ```
  from dataclasses import dataclass

  @dataclass
  class Variant:
      chrom: str
      pos: int
      ref: str
      alt: str
      qual: float = 0.0
  ```

  You get `__init__`, `__repr__`, and equality for free, with attribute access (`v.pos`) instead of `v["pos"]`. Prefer these over parallel lists or dicts-of-fields for anything with a fixed shape.
- **Type hints.** Annotations like `def parse_vcf(path: Path) -> list[Variant]:` are not enforced at runtime, but tools (`mypy`, Pyright, your editor) use them to catch bugs before you run the code, and they're the best function documentation you can write. Start with hints on function signatures and dataclass fields — don't try to annotate everything at once.
- **Context managers beyond `open()`.** `with pysam.AlignmentFile(...) as bam:`, `with tempfile.TemporaryDirectory() as d:`, `with contextlib.suppress(FileNotFoundError):`. Any

resource that needs deterministic cleanup — file handles, temp dirs, database cursors, locks — is a candidate.

Before adding a dependency, check `collections`, `itertools`, `functools`, `pathlib`, `dataclasses`, `contextlib`, `statistics`. The Python standard library solves most problems you'll hit in genomics scripting, and stdlib code ages well — no surprise breaking changes when your analysis reviewer tries to rerun it two years later.

You wrote a function with a type-hinted signature, a dataclass for one biological record, and a `pathlib`-based file iteration. This is Python that will read well in a pull request.

You already know `awk '{print $1}'` extracts column 1. The Python equivalent: `line.split('\t')[0]`. Your Unix skills transfer directly. When in doubt, prototype in awk, then scale up in Python.

Block 4: Pandas, Plotting & Bio Libraries

Sprint 5: Pandas for Tabular Genomic Data *(25 min)*

- Load GWAS summary stats: `df = pd.read_csv('sumstats.tsv', sep='\t', dtype={'CHR': str})`
- Filter: `sig = df[df['P'] < 5e-8]` — genome-wide significant hits
- GroupBy: `df.groupby('CHR')['P'].min()` — best p-value per chromosome
- Merge: `pd.merge(genotypes, phenotypes, on='SAMPLE_ID')` — the most common GWAS operation

You loaded, filtered, and grouped real GWAS data. This replaces hours of spreadsheet work.

Sprint 6: Visualization *(25 min)*

- Manhattan plot: scatter plot of $-\log_{10}(P)$ vs. genomic position, colored by chromosome
- QQ plot: expected vs. observed $-\log_{10}(P)$ — the standard GWAS diagnostic
- Use matplotlib for control, seaborn for aesthetics. Always label axes, always include titles.
- Save to file: `plt.savefig('manhattan.png', dpi=150, bbox_inches='tight')`

You made a Manhattan plot from real data. Screenshot it for your portfolio.

Sprint 7: NumPy + SciPy = Fast Math *(25 min)*

- NumPy arrays: 100× faster than Python lists for math. `np.mean(quality_scores)` instead of `sum()/len()`

- Vectorized operations: never loop over millions of variants. `genotypes * 2` multiplies every element at once.
- SciPy stats: `scipy.stats.ttest_ind(group1, group2)` for quick hypothesis tests
- Multiple testing: `from statsmodels.stats.multitest import multipletests` — you'll need this for any -omics analysis

You ran a statistical test on biological data in Python. No GUI, no spreadsheets — just code.

Sprint 8: Bio Libraries: pysam, Biopython, scanpy *(25 min)*

- pysam: read BAM files programmatically. Count reads mapping to a gene in 3 lines.
- Biopython: `SeqIO.parse()` handles FASTA, FASTQ, GenBank. `Entrez.efetch()` queries NCBI.
- scanpy: the single-cell analysis framework. We'll use it heavily in Blocks 9–10.
- scikit-allel: population genetics in Python — allele frequencies, LD, PCA from VCF data

You queried NCBI from Python and read a BAM file programmatically. You're automating research.

End of Blocks 3–4

You just absorbed a new language AND applied it to biology. Take some time off. Let it consolidate.

Checkpoint:

- You can parse any common genomic file in Python (FASTA, BED, VCF, TSV)
- You can use Pandas to load, filter, group, and merge tabular data
- You can make a Manhattan plot and a QQ plot
- You understand why vectorized NumPy operations are faster than Python loops

Resources

- McKinney – *Python for Data Analysis* (the Pandas bible)
- VanderPlas – *Python Data Science Handbook* (free online)
- Real Python (realpython.com) — searchable tutorials for anything

Blocks 5–6: R, Bioconductor & Differential Expression

Primary: Bioconductor, differential expression, ggplot2 | **Interleave:** Statistics concepts, Unix file prep

R has Bioconductor — thousands of biology-specific packages with nothing comparable in Python. DESeq2, edgeR, GenomicRanges, Seurat all live here. You need both languages.

R is not installed by default on macOS or most Linux distributions. Download the pre-built binary for your OS/architecture from CRAN (https://cran.r-project.org), then install RStudio Desktop for a comfortable IDE (https://posit.co/download/rstudio-desktop/). See Appendix A for more.

Block 5: R Basics + Bioconductor

Sprint 1: R in 25 Minutes — Vectors, Lists, Data Frames *(25 min)*

- **Vectors.** `x <- c(1, 2, 3); mean(x)`. Everything in R is a vector — even a single number is a length-1 vector. Vectors are *homogeneous*: one type throughout (numeric, character, logical).
- **Lists.** Heterogeneous containers:

  ```
  res <- list(pval = 0.003,
              estimate = 1.42,
              samples = c("A", "B", "C"))
  res$pval # 0.003
  res[["samples"]] # c("A", "B", "C")
  ```

  Access by name with `$` or `[[`. Every complex R object — `lm()` output, a `DESeqDataSet`, a phylo tree — is a list underneath. Learn `str(obj)` now; you'll use it constantly to dig into unfamiliar objects.
- **Data frames.** A list of equal-length columns with row-and-column indexing: `df <- read.csv('samples.tsv', sep='\t')`. First three commands on any new data frame: `head(df)`, `names(df)`, `str(df)`.
- **Indexing gotchas.** R is 1-based: `df[1,]` is row 1, not row 2. Negative indices *exclude*: `df[-1,]` drops row 1. Logical vectors filter: `df[df$pval < 0.05, ]`.

You loaded a TSV, inspected its structure with `str()`, pulled one column and one row, and know the difference between a vector, a list, and a data frame. Apply functions and the tidyverse come next.

Sprint 2: Lists, the Apply Family & Functional R *(25 min)*

R grew up as a functional language:

instead of writing for-loops, you hand a function to another function that runs it over a collection. This pattern is everywhere in R code — you need to read it fluently even if you'd rather write loops.

- **`lapply()` — run f over a list, return a list.**

  ```
  genes <- list(BRCA1 = counts1, BRCA2 = counts2)
  means <- lapply(genes, mean)
  # means$BRCA1, means$BRCA2
  ```

- **`sapply()`** does the same and simplifies to a vector/matrix when it can. Convenient interactively; prefer `vapply()` in scripts — it type-checks the return shape and fails loudly when a single element deviates.
- **`apply()` — walk rows or columns of a matrix.**

  ```
  row_means <- apply(counts_matrix, 1, mean)
  col_sds <- apply(counts_matrix, 2, sd)
  ```

  First arg = matrix; `1` = rows, `2` = columns; third arg = the function.
- **`mapply()` / `Map()` — over multiple parallel inputs.** `mapply(t.test, group_a_list, group_b_list)` runs a t-test pairing element 1 of each, element 2 of each, etc.
- **`tapply()` — split by a grouping vector, then apply.** `tapply(expression, tissue, mean)` is "mean expression per tissue" in one call.
- **Why this matters.** DESeq2 results come back as a list; `apply()` is how you compute per-gene statistics; `lapply()` appears in every production R pipeline. Loops are not wrong, but apply-family code is shorter and more idiomatic.
- **The modern equivalent.** The `purrr` package (part of the tidyverse) gives you `map()`, `map_dbl()`, `map_chr()`, `map2()`, `pmap()` — same idea, stricter return types. We'll use `purrr::map_dbl()` in the next sprint.

> You computed per-gene means and per-sample standard deviations on a count matrix without writing a single explicit loop. This is the R way.

Sprint 3: Tidyverse as a Modern Layer

(25 min) Base R works, but routine data wrangling is verbose. The tidyverse is a family of packages (`dplyr`, `tidyr`, `readr`, `purrr`, `ggplot2`, `stringr`, `forcats`, ...) that share a consistent grammar: pipe a data frame through a sequence of *verbs*, each returning a modified data frame. Once you internalize the verbs, most analyses become one-glance readable.

Install once with `install.packages("tidyverse")` in an R session — that single call pulls in every package listed above. Docs: https://www.tidyverse.org.

- **The pipe.** `%>%` (from `magrittr`, re-exported by dplyr) and the newer base-R `|>` (R $\geq$ 4.1) both do the same thing: pass the left-hand side as the first argument of the right-hand side. They're Unix `|` for data frames.
- **The five verbs you'll use constantly.**

  ```
  library(dplyr)
  de_results |>
    filter(padj < 0.05) |>
    mutate(direction = if_else(log2FC > 0,
                               "up", "down")) |>
    group_by(chromosome, direction) |>
    summarise(n = n(), .groups = "drop") |>
    arrange(desc(n))
  ```

`filter` picks rows; `mutate` adds/changes columns; `group_by` + `summarise` compute per-group statistics; `arrange` sorts; `select` picks columns. Five verbs cover 80% of real analysis.

- **Tibbles.** `tibble()` creates a modern data frame that prints neatly (no auto-factor conversion, no surprising type coercion). `readr::read_csv()` returns a tibble; it behaves like a `data.frame` everywhere else.
- **Reshaping.** `pivot_longer()` and `pivot_wider()` (`tidyr`) go between wide and long formats — critical for plotting in ggplot2 and for joining disparate datasets.
- **Joins.** `inner_join()`, `left_join()`, `anti_join()`. Merge GWAS summary stats with gene annotations by SNP ID in one line.
- **`across()` for column-wise operations.** `mutate(across(starts_with("sample_"), log1p))` log-transforms every sample column at once.

Tidyverse is great for interactive analysis and scripts humans will read. Package authors often prefer base R internally — it has zero dependencies and a more stable API. Both styles are valid; most real projects mix them.

You chained five dplyr verbs to go from raw DE results to a ranked table of up/down counts per chromosome. Compare the equivalent base-R code (`aggregate`, nested `[`) — same output, very different ergonomics.

Sprint 4: Bioconductor: GRanges = Genomic Intervals *(25 min)*

- Install: `BiocManager::install('GenomicRanges')`
- GRanges: `gr <- GRanges(seqnames='chr1', ranges=IRanges(start=100, end=200))`
- `findOverlaps(peaks, genes)` — which peaks overlap which genes? This replaces `bedtools intersect`.
- `rtracklayer::import('file.bed')` — load BED, GFF, BigWig directly into R

You performed a genomic interval overlap in R. Same concept as bedtools, different syntax.

Sprint 5: SummarizedExperiment: The Container That Holds Everything *(25 min)*

- SummarizedExperiment = counts matrix + gene info (rowData) + sample info (colData)
- Every Bioconductor analysis tool expects data in this format. Learn it once, use it everywhere.
- Load a real dataset: recount3 or GEOquery to pull RNA-seq data directly from public repositories
- Explore: `assay(se)[1:5, 1:5]` — the first 5 genes and 5 samples

You loaded a real RNA-seq dataset from a public database into R. Data is ready for analysis.

Block 6: Differential Expression + ggplot2

Sprint 6: DESeq2: The Gold Standard DE Analysis *(50 min)*

- Build a DESeqDataSet from your SummarizedExperiment + a design formula (`~condition`)
- Run: `dds <- DESeq(dds); res <- results(dds)` — the entire analysis in two lines
- Understand what happened: normalization (size factors), dispersion estimation (empirical Bayes shrinkage), NB GLM fitting, Wald test
- Extract results: `res[order(res$padj), ]` — genes sorted by significance
- Make a volcano plot: plot $-\log_{10}$(pvalue) vs. $\log_2$(FoldChange) with ggplot2

You ran a full differential expression analysis on real data. This is publishable work.

Sprint 7: ggplot2: Grammar of Graphics *(25 min)*

- The formula: `ggplot(data, aes(x, y)) + geom_point() + theme_minimal()`
- Layers: `+ geom_violin() + geom_boxplot(width=0.1)` — combine geoms for rich plots
- Facets: `+ facet_wrap(~chromosome)` — split by a variable automatically
- Save: `ggsave('figure1.pdf', width=8, height=6)` — publication-ready output

You created a multi-panel publication figure. This goes in your portfolio.

Sprint 8: Compare DE Methods (Interleaving!) *(25 min)*

- Run the same dataset through limma-voom: `voom(counts) + lmFit + eBayes + topTable`
- Compare results: how many genes are significant in DESeq2 but not limma? Vice versa?
- Make a Venn diagram or UpSet plot of the overlap — this is how real papers compare methods
- Think about why: DESeq2 and limma make different statistical assumptions. Which matter for your data?

You compared two statistical methods on the same data and thought critically about why results differ.

End of Blocks 5–6

You now have working fluency in bash, Python, and R, and can run a differential expression analysis. Take a break before moving on.

Checkpoint:

- You can manipulate data in R with tidyverse
- You understand GRanges and SummarizedExperiment
- You can run a DESeq2 analysis end-to-end
- You can make publication-quality ggplot2 figures
- You can explain what empirical Bayes shrinkage does and why DESeq2 uses it

Resources

- R for Data Science, 2nd Ed (r4ds.hadley.nz) — free online, excellent
- DESeq2 vignette (follow the Beginner's Guide first, then the full vignette)
- StatQuest YouTube: the DESeq2 and edgeR episodes are outstanding

- Bioconductor workflows: bioconductor.org/packages/release/workflows/

Blocks 7–8: Statistics for Genomics

Primary: Distributions, multiple testing, linear models, PCA | **Interleave:** Python + R implementation, genomic data

Every bad paper in genomics has a statistics problem, not a coding problem.
You don't need to derive proofs. You DO need to know: which distribution fits my data? What test is appropriate? What happens if my assumptions are wrong?
We'll learn by simulating — generate fake data, break assumptions, see what happens.

Block 7: Distributions, Testing & Models

Sprint 1: What Distribution Is My Data? *(25 min)*

- Simulate each: `np.random.binomial`, `np.random.poisson`, `np.random.negative_binomial`, `np.random.normal`
- Plot histograms of each side by side. Now overlay a real dataset (RNA-seq counts, allele frequencies, expression values)
- Key insight: RNA-seq counts are NOT normal. They're negative binomial. This is why DESeq2 uses NB models.

You can look at a histogram and identify the distribution family. That's statistical intuition.

- Binomial: counting successes (alt allele reads out of total reads)
- Poisson: rare events (mutations per gene)
- Negative binomial: counts with extra variation (RNA-seq — this is the big one)
- Normal: continuous measurements after transformation (log-expression, quantitative traits)
- Beta: values between 0 and 1 (allele frequencies, methylation)
- Hypergeometric: overlap significance (gene set enrichment, Fisher's exact test)

Sprint 2: Multiple Testing: The Genomics Problem *(25 min)*

- Simulate 1,000,000 null p-values: `p = np.random.uniform(0, 1, 1_000_000)`
- How many are < 0.05? About 50,000. That's 50,000 false positives from pure noise.
- Apply Bonferroni: threshold $= 0.05/1{,}000{,}000$. Very conservative, misses real signal.
- Apply Benjamini–Hochberg: controls false discovery RATE, not family-wise error. This is the genomics standard.
- Make a QQ plot: sorted $-\log_{10}(\text{observed } P)$ vs. $-\log_{10}(\text{expected } P)$. Deviation = signal.

You saw with your own eyes why $p < 0.05$ fails at genomic scale.

Sprint 3: Linear Models: The GWAS Engine *(25 min)*

- A GWAS is just a linear regression: phenotype $\sim$ genotype + covariates, run millions of times
- In Python: statsmodels OLS. In R: `lm()`. Fit a model, interpret coefficients, check residuals.
- GLMs: logistic regression for case/control (binary outcomes), Poisson/NB for count outcomes
- Mixed-effects models: add random effects for family structure or batch — conceptual understanding is enough for now

You fit a linear model to genetic data — a single-SNP association test, which is the core unit of a GWAS.

- Using a t-test when you have covariates to control for (use regression instead)
- Treating FDR as the probability any specific result is false (it's the expected proportion)
- Ignoring population structure in GWAS (this inflates false positives massively)
- Applying normal-distribution tests to count data (use NB or nonparametric tests)

Block 8: Dimensionality Reduction & Bayesian Ideas

Sprint 4: PCA: See the Hidden Structure *(25 min)*

- Run PCA on 1000 Genomes data (or a pre-computed matrix): `sklearn.decomposition.PCA`
- Plot PC1 vs PC2. Color by population. The clusters ARE population structure.
- Scree plot: how many PCs capture real signal vs. noise? The 'elbow' tells you.
- This is why GWAS includes PCs as covariates: they capture ancestry and prevent false associations.

You see population structure emerge from genotype data. PCA is the most-used tool in genomics.

Sprint 5: UMAP + Clustering (for Single-Cell) *(25 min)*

- UMAP: nonlinear dimensionality reduction. Useful for single-cell, but easy to over-interpret.
- Key rule: UMAP preserves local structure, not global distances. Clusters close together aren't necessarily similar.
- Leiden clustering: graph-based, resolution-dependent. Try multiple resolutions and compare.
- Practical: load a pre-processed single-cell dataset and make UMAP + cluster plots in scanpy or Seurat

You made a UMAP plot of single-cell data and critically evaluated what it does and doesn't tell you.

Sprint 6: Bayesian Thinking (Conceptual) *(25 min)*

- Bayes' theorem in one sentence: update your beliefs with new data. Prior × Likelihood → Posterior.
- Empirical Bayes: learn the prior from the data itself. This is how DESeq2 shrinks dispersion estimates.
- Why it matters: Bayesian fine-mapping (SuSiE) gives you credible sets of causal variants, not just p-values.
- You don't need to implement MCMC. You DO need to understand what "95% credible set" means.

You can explain Bayes' theorem with a genomics example. Conceptual understanding > mathematical derivation.

End of Blocks 7–8

This is the most conceptually dense part of the curriculum. Take some time to let it settle before moving into applications.

Checkpoint:

- You can identify which distribution fits a genomic dataset
- You can apply and explain Bonferroni and BH correction
- You can fit a linear model and interpret coefficients
- You can run PCA and explain what population structure looks like
- You can explain empirical Bayes at a conceptual level

Resources

- StatQuest YouTube (Josh Starmer) — highly recommended for visual explanations of statistical concepts
- Holmes & Huber – *Modern Statistics for Modern Biology* (free online)
- Storey & Tibshirani – Statistical significance for genomewide studies (the FDR paper)

Blocks 9–10: Applications — Single-Cell & GWAS

Primary: scRNA-seq pipeline, GWAS pipeline | **Interleave:** All previous skills converge here

Everything you learned — Unix, Python, R, statistics — comes together in these two blocks. You'll run complete real-world analyses from raw data to biological insight.

Block 9: Single-Cell RNA-seq — Start to Finish

Sprint 1: Technology + Data Orientation *(25 min)*

- 10x Chromium captures RNA from individual cells → UMI count matrix (genes × cells).
- Download a public PBMC dataset from the 10x Genomics website (https://www.10xgenomics.com/datasets); a pre-processed h5ad is fine to start.
- Install scanpy if you haven't: `mamba install -c conda-forge -c bioconda scanpy`. Docs: https://scanpy.readthedocs.io/. Load: `adata = sc.read_h5ad('pbmc.h5ad')`.
- Explore: `adata.shape`, `adata.obs.head()`, `adata.var.head()`
- Think: what does each row represent? Each column? What are the expected ranges for counts?

You have real single-cell data loaded and you understand its structure.

Sprint 2: QC — Find and Remove the Junk *(25 min)*

- Calculate QC metrics:

  ```
  sc.pp.calculate_qc_metrics(adata, qc_vars=['mt'])
  ```

  This adds `n_genes_by_counts`, `total_counts`, and `pct_counts_mt` to `adata.obs`.
- Plot distributions:

  ```
  sc.pl.violin(adata,
    ['n_genes_by_counts', 'total_counts', 'pct_counts_mt'])
  ```
- Filter: keep cells with 200–5000 genes and <20% mito. These thresholds have biological reasons.
- Doublet detection with Scrublet: `sc.pp.scrublet(adata)` or `sc.external.pp.scrublet(adata)` depending on scanpy version

You filtered out low-quality cells and doublets. The data is now clean.

Sprint 3: Normalize → HVGs → PCA → UMAP → Cluster *(25 min)*

- `sc.pp.normalize_total(adata); sc.pp.log1p(adata)` — library size normalization
- `sc.pp.highly_variable_genes(adata)` — select ∼2000 informative genes
- Run the pipeline:

```
sc.tl.pca(adata)
sc.tl.neighbors(adata)
sc.tl.umap(adata)
sc.tl.leiden(adata, resolution=0.5)
```

- `sc.pl.umap(adata, color=['leiden'])` — a single-cell cluster plot from raw data

You went from raw counts to clustered UMAP in 6 lines of code.

Sprint 4: Annotate Cell Types + Differential Expression *(25 min)*

- Plot known markers: `sc.pl.dotplot(adata, ['CD3D', 'CD14', 'MS4A1', 'NKG7'], groupby='leiden')`
- Assign labels:

```
adata.obs['cell_type'] = adata.obs['leiden'].map(
  {0: 'CD4 T', 1: 'Monocytes', ...})
```

- Pseudobulk DE (the right way): aggregate counts per donor×cell_type, then use DESeq2 in R
- Why pseudobulk? Cell-level tests inflate p-values because cells from the same donor aren't independent.

You annotated cell types using marker genes and understand why pseudobulk DE matters. This is current best practice.

- CITE-seq: same cells, RNA + protein measurements → Seurat WNN for joint embedding
- scATAC-seq: chromatin accessibility per cell → ArchR, Signac
- Spatial transcriptomics: gene expression + spatial location → Squidpy, cell2location
- Perturb-seq: CRISPR perturbations linked to transcriptomes → Mixscape, scMAGeCK

Block 10: GWAS Pipeline — Variant to Function

Sprint 5: Genotype QC with PLINK2 *(25 min)*

- PLINK2 is the workhorse. Install via bioconda (`mamba install -c bioconda plink2`) or download a pre-built binary from https://www.cog-genomics.org/plink/2.0/. Get a small dataset (HapMap or 1KG subset in PLINK format).
- Sample QC: `-mind` (call rate), `-het` (heterozygosity), `-check-sex`, `-king-cutoff` (relatedness)
- Variant QC: `-geno` (call rate), `-hwe` (Hardy–Weinberg), `-maf` (minor allele frequency)
- PCA: `plink2 -pca 10` — compute population structure. Plot PC1 vs PC2.

You QC'd a genotype dataset. Every GWAS starts here.

Sprint 6: Association Testing *(25 min)*

- Simulate a phenotype correlated with genotype (so you know the true answer)
- Run: `plink2 -glm ...` — this fits a linear (or logistic) model at every variant
- For real data: use REGENIE or BOLT-LMM (mixed models that handle population structure and relatedness)
- Make a Manhattan plot and QQ plot from the results. Check λ_{GC} for inflation.

You ran a GWAS and made the diagnostic plots. You can tell if results are inflated or clean.

Sprint 7: Post-GWAS: Heritability, Fine-Mapping, Interpretation *(25 min)*

- LDSC (`https://github.com/bulik/ldsc`): estimate SNP-heritability from summary statistics. How much of the trait is explained by common variants? Install via conda with a Python 2.7 env or follow the repo instructions.
- Fine-mapping: the SuSiE R package (`https://stephenslab.github.io/susieR/`) gives you 95% credible sets — the smallest set of variants likely to contain the causal one. `install.packages("susieR")`.
- Colocalization: the `coloc` R package (`install.packages("coloc")`, `https://chr1swallace.github.io/coloc/`) asks whether your GWAS signal shares a causal variant with a nearby eQTL.
- PRS (polygenic risk scores): aggregate small effects across the genome into a single predictor

You traced a GWAS signal from association to potential causal variant to gene.

End of Blocks 9–10

You've now run the two most important analysis pipelines in modern genomics end-to-end. Take a break before the final unit.

Checkpoint:

- You can run a complete scRNA-seq analysis from raw counts to annotated clusters
- You can explain every step in the single-cell pipeline and why each matters
- You can QC genotype data, run an association test, and make Manhattan/QQ plots
- You can explain heritability, fine-mapping, and colocalization at a conceptual level

Resources

- Scanpy tutorials (scanpy.readthedocs.io) — the PBMC 3k tutorial is the canonical starting point
- Heumos et al. – Best practices for single-cell analysis (Nat Rev Genet, 2023)
- Uffelmann et al. – Genome-wide association studies (Nat Rev Methods Primers, 2021)
- Marees et al. – A tutorial on conducting GWAS (step-by-step, very practical)
- LDSC wiki on GitHub — the definitive usage guide

Blocks 11–12: AI, Right Questions & Capstone

Primary: AI prompting, data expectations, reproducible pipelines, capstone project

> Tools change. Methods change. The ability to ask the right question, recognize unexpected data, and evaluate AI output — that lasts your whole career.

Block 11: Asking the Right Questions + AI Mastery

Sprint 1: The Question Framework *(25 min)*

- Before EVERY analysis, write down answers to: What's the biological question? What would a yes look like? What would a no look like?
- Data inventory: how many rows? columns? What does each represent? What are the expected ranges?
- Sanity checks: ∼4–5M variants per human genome, ∼20K protein-coding genes, scRNA-seq: 500–5000 genes/cell
- Red flags to watch for: flat Manhattan plot (no signal), bimodal quality scores, batch-correlated PCA

> You wrote a pre-analysis checklist for a dataset. This prevents weeks of wasted analysis on broken data.

The Question Map

Before you start	What is the question? What would a definitive answer look like?
Loading data	What do rows/columns represent? Expected dimensions? Missing values?
Choosing methods	What does this method assume? Does my data meet the assumptions?
After QC	Do distributions look right? Do controls work? Any batch effects?
Interpreting results	Is the effect size meaningful? Could confounders explain this?
Before publishing	Can someone else reproduce this? Are all parameters recorded?

Sprint 2: Prompting AI Like a Bioinformatician *(25 min)*

- Context-first prompting: "I have a 10x scRNA-seq h5ad file, 50K cells, human PBMCs. I want to..."
- Constraint prompting: "Use scanpy 1.9+, avoid deprecated functions, include comments at each QC threshold"

- Verification prompting: "Include assertions that check the output dimensions and warn if something unexpected happens"
- Debugging: paste the FULL error message + stack trace. Never describe the error in your own words.

You wrote a prompt that generated working, well-documented bioinformatics code on the first try.

Sprint 3: Spotting AI Hallucinations *(25 min)*

- Common AI mistakes: inventing function names, confusing package versions, incorrect parameter names
- Always test: run AI code on a small known-answer dataset before trusting it on real data
- Cross-validate: ask two different AI tools the same question. If they disagree, investigate.
- Version trap: AI may mix up scanpy 1.8 vs 1.10 syntax, or old Seurat v3 vs v5 functions

You found and corrected an AI hallucination. This is a critical skill for AI-augmented research.

- Always run AI-generated code before trusting it.
- Cite AI assistance in your methods section — transparency matters.
- Never use AI for novel method development without deep verification.
- AI is a study partner, not a replacement for understanding.

Block 12: Reproducible Pipeline + Capstone

Sprint 4: Snakemake: Your Pipeline in Code *(50 min)*

- Install: `mamba install -c bioconda -c conda-forge snakemake`. Docs: https://snakemake.readthedocs.io/.
- Snakemake defines a pipeline as a set of rules. Each rule: input files → output files + a shell/Python command.
- Start simple: rule download + rule qc + rule align. Run with `snakemake -cores 4`.
- Snakemake automatically figures out the execution order. Change an input file? It re-runs only what's needed.
- Add a `conda:` directive to each rule for per-step environments. Now it's fully reproducible.
- Alternative: Nextflow + nf-core if you prefer. Install Nextflow with `curl -s https://get.nextflow.io | bash`. nf-co.re (https://nf-co.re) has community-curated pipelines for most assays.

You have a multi-step Snakemake pipeline that someone else could clone and run.

Capstone Project

This is your final project. It ties together everything from the past 11 blocks into a single, complete analysis.

- Choose a biological question and a public dataset (GEO, SRA, GWAS Catalog).
- Build a reproducible pipeline (Snakemake or Nextflow) in a Git repository.
- Analyze with Python and/or R, using appropriate statistical methods.
- Produce at least 3 publication-quality figures with captions.
- Write a 5–10 page report: question, methods, results, interpretation.
- Include an AI journal: how you used AI tools, what worked, what didn't.

- Complete Problem Set 4 (QTL Mapping) from the Running Project — this is the recommended capstone if you've been working through Problem Sets 1–3
- Reproduce a published scRNA-seq analysis with updated methods (new normalization, better clustering)
- GWAS-to-function: fine-map a locus, colocalize with eQTLs, identify candidate genes
- Cross-ancestry PRS: build polygenic risk scores, test portability across populations
- CRISPR screen reanalysis: apply MAGeCK + BAGEL2 to a published dataset, compare hit lists

Sprint 5: The Reproducibility Test *(25 min)*

- Clone your repo to a different computer (or ask a friend/AI to try).
- Can they install the environment and run the pipeline end-to-end?
- If yes: you're done. If no: fix what's broken. This is the real test.

Someone else can reproduce your analysis from your repo. That is the standard to aim for.

End of the curriculum

At this point you can process genomic data in Unix, analyze it in Python and R, apply rigorous statistics, and produce reproducible research. The foundation is in place — continued learning comes from applying these skills to real research questions.

Running Project: From Growth Curves to QTL Mapping

This four-part project follows a single scientific narrative: can you find the genetic basis of a yeast strain's response to an unknown compound? Each problem set is designed to be worked on during or shortly after the block that teaches the relevant skills. The problems build on each other, so work through them in order.

All data can be found in the GitHub repository `joshsbloom/QuantGenSandbox` and on the linked Dropbox. The project culminates in a QTL mapping scan that identifies a candidate gene and generates a hypothesis about the compound's identity.

1. Yeast Growth Curves	Blocks 5–6 (R)	R scripting, ggplot2, t-tests, permutation tests, power simulation
2. Heritability & Mixed Models	Blocks 7–8 (Stats)	Linear models, random effects, heritability, experimental design
3. Short-Read Alignment & Strain ID	Blocks 9–10 (Apps)	Unix pipelines, bwa, sambamba, GATK, BLAST, Hamming distance
4. QTL Mapping	Blocks 11–12 (Capstone)	Genome-wide scans, permutation thresholds, LOD scores, confidence intervals

Problem Set 1: Yeast Growth Curves

Work on this after Blocks 5–6.

Two yeast strains — BY (lab reference) and RM (vineyard isolate) — are grown in YNB + glucose media. You want to determine whether they grow at different rates. 120 µL cultures of each strain were grown overnight to saturation in a 96-well plate (Plate080525_YNB_GLU): BY in well A01, RM in well A02. You will inoculate replicate growth curves using a Biomek liquid-handling robot and then analyze the resulting data.

Part A: Generate the Biomek Randomization CSV

The Biomek robot needs a CSV file specifying: which source well to pipette from, which target well to pipette into, and how much volume. The 96-well plate has rows A–H and columns 01–12. All column numbers less than 10 must have a leading zero (e.g., B03, C09).

Goal: Inoculate 1 µL of saturated culture into 30 random wells for each strain, giving you 30 replicate growth curves per strain. Edge wells (row A, row H, column 01, column 12) should be left as media blanks to control for evaporation and contamination.

The CSV must be comma-delimited with DOS line endings and this header:

```
Source.Plate,Source.Wells,Strain,Target.Plate,Target.Wells,Volume
```

- Write R code to generate the Biomek CSV file. Use `set.seed(10)` for reproducibility.
- Useful functions: `data.frame()`, `sample()`, `paste0()`, `rep()`, `toupper()`, `sprintf()`, `write.table()` with `eol='\r\n'`.
- Challenge: write the entire script in 4 lines of R.

Part B: Fit Growth Curves & Extract Doubling Times

Read in the plate reader data (`growthCurves_01.csv`) and the Biomek CSV (`randomize_01.csv`) from the `joshsbloom/QuantGenSandbox` GitHub repository, under `projects/growthCurves/`.

- Use the QurvE R package (`growth.gcFitSpline` function) to fit spline models to the growth curves.
- Use lubridate to convert time stamps into decimal hours.
- Extract doubling times during log-phase growth from the fitted models.
- Visualize a few individual growth curves with their fits to confirm the procedure works and the fitting looks reasonable.

Part C: Visualize & Test for Differences

- Use ggplot2 to make a histogram of doubling times by strain (colored by strain).
- Make a violin plot showing individual data points, mean, and 95% confidence interval of the mean for each strain.
- Use `t.test()` to test for a difference in means. Think about whether a paired t-test is appropriate here and why or why not.

Part D: Permutation Test

The assumptions of a parametric t-test may not hold. Perform a permutation test to assess significance without distributional assumptions.

- Generate a null distribution by permuting strain labels 1000 times. For each permutation, compute the t-statistic.
- Visualize the null distribution and mark where the observed (unpermuted) t-statistic falls.
- Calculate a permutation-based p-value.
- Question: What assumptions does the permutation test make? What assumptions of the parametric t-test might be violated here?

Part E: Power Analysis by Simulation

How small a fitness difference can you detect? Evaluate statistical power across a grid of parameters:

- Replicates per strain: 3, 6, 12, 24, 48, 96, 384
- Doubling time means for the new strain: 60 to 240 minutes in 3-minute increments
- Assume BY mean = 90 min, SD = 18 min; the other strain has the same SD

- For each combination: simulate 1000 experiments from two normal distributions with the appropriate means, SD, and sample size.
- Power = fraction of simulations with t-test $p < 0.05$.
- Plot with ggplot2: x = difference in doubling time, y = power, colored by sample size (as a factor).
- Question: If you want to detect a 10-minute difference in doubling time, how many replicates do you need?

Problem Set 2: Heritability & Mixed Models

Work on this after Blocks 7–8.

An unknown compound, chemical X, may inhibit fungal growth. If different yeast strains respond differently, the trait may be heritable, and you could use QTL mapping to find the genetic basis. Eight lab members set up an experiment with ten yeast isolates in YPD and YPD + chemical X across four 96-well plates. A power outage left you with only endpoint OD readings.

Part A: Read & Merge Data

- Read the design file (`02_OD_design.csv`) and the plateR-formatted endpoint OD data (`02_OD_measurements.csv`) from `joshsbloom/QuantGenSandbox` under `projects/modeling-FixedRandom/`.
- Use the plateR R package to parse the plate data.
- Merge measurements with design into a single data frame.

Part B: Visualize

- Make 8×12 heatmaps of OD for each plate using ggplot2 with a viridis color scale. Are there plate- or position-dependent effects?
- Make additional heatmaps showing experimenter and treatment by well position.
- Create a publication-ready 3-panel figure (using patchwork) with panel A twice the size of B and C combined:
- Panel A: Violin plots of endpoint OD by strain with separate violins per treatment. Overlay jittered points, show means, and draw lines connecting YPD vs. YPD:X means within each strain.
- Panel B: Violin + jitter plots grouped by experimenter.
- Panel C: Violin + jitter plots grouped by plate.
- Questions: What do you conclude about strain, treatment, experimenter, and plate effects? Why was it problematic to have different people with different pipettes add strain and media?

Part C: Fixed-Effects Linear Models

- Fit: `OD ~ strain + treatment + plate + experimenter`. What is the average treatment effect? Use `Anova()` to test each factor.
- Add an interaction: `OD ~ strain + treatment + strain:treatment + plate + experimenter`.
- Use a likelihood ratio test to determine whether the interaction term is justified.
- Compare these results to your visualization in Part B.

Part D: Broad-Sense Heritability

- Restrict to YPD:X wells. Estimate broad-sense heritability: $H^2 = \text{Var(Strain)}/(\text{Var(Strain)} + \text{Var(Residual)})$.
- First ignoring experimenter and plate, then including them.
- How do the estimates differ? Why does accounting for these nuisance variables change the heritability estimate?
- Why is heritability of this trait hard to interpret on its own?

Part E: Mixed Model with Random Effects

- Fit: `OD ~ strain * treatment + (1|plate) + (1|experimenter)`. Use lme4 or a similar package.
- Compare to the fixed-effects model using BIC.
- Why are plate and experimenter better modeled as random effects here? Think beyond just the BIC comparison.

Part F: Normalized Response to Chemical X

- Get residuals from a model of OD with random effects of plate and experimenter.
- Compute mean residuals for each strain in YPD.
- Subtract each strain's YPD mean from its residuals in YPD:X. Retain only these corrected YPD:X values.
- Make a violin plot of these normalized strain responses with jittered data points.
- Question: What does this normalization accomplish? What do you conclude about strain-level sensitivity to chemical X?

Part G: Heritability of the Normalized Trait

- Estimate H^2 for the normalized trait.
- Reflect: how can heritability be misleading? How would the results from Parts F and G guide a QTL mapping experiment?

Problem Set 3: Short-Read Alignment & Strain Identification

Work on this after Blocks 9–10.

The lab decided to cross BY to one of the wild isolates and use QTL mapping. To do this, HO endonuclease was disabled in each parent (to maintain haploid state), the strains were mated, the resulting diploid was sporulated, and Heriberto tetrad-dissected over 100 tetrads. DNA sequencing libraries were prepared for each segregant and for the unknown wild parent strain. Unfortunately, no one recorded which wild isolate was used. Your job is to figure it out from the sequencing data.

Data files (FASTQ for the mystery strain and segregants, reference genome, and known variant sites) are available on the linked Dropbox.

Part A: Identify the Mystery Strain

- Manually BLAST a few short reads from the mystery strain FASTQ files against NCBI to confirm they come from yeast.
- Use `bwa mem` to align the paired-end reads to `sacCer3.fasta`.
- Pipe the output through sambamba to convert SAM to BAM and sort.
- Visualize the alignments in IGV. Look at the first 10–20 Kbp on ChrI. What do you see? Does it make sense?
- Use GATK ASEReadCounter with the known variant sites (`mega_filtered.vcf.gz`) to count reference vs. alternate alleles at each site.
- In R: classify sites as alternate (1) or reference (0) based on which allele has more reads.
- Calculate the Hamming distance between the mystery strain's genotypes and each wild isolate in `sparseMatrixGeno1011.RDS`.
- Question: What is the mystery strain?

Part B: Visualize Meiotic Crossovers

- Align reads from the first segregant (seg1 FASTQ files) using the same pipeline.
- Retain genotype calls only at variant sites distinguishing BY from the identified mystery strain.
- Recode: BY variants = 0, mystery strain variants = +1.
- Visualize variant inheritance across the genome.
- By eye: how many crossovers do you see per chromosome?

The following commands chain alignment, format conversion, sorting, and variant counting:

```
bwa index sacCer3.fasta

bwa mem -t 8 sacCer3.fasta \
  reads_r1.fq.gz reads_r2.fq.gz \
  | sambamba view -S -f bam /dev/stdin \
  | sambamba sort -o output.bam /dev/stdin

gatk ASEReadCounter -R sacCer3.fasta \
  -I output.bam -V mega_filtered.vcf.gz \
  -O counts.table
```

Problem Set 4: QTL Mapping

Work on this during Blocks 11–12 as your capstone, or alongside the capstone.

You now have ∼100 haploid recombinant segregants from a cross of BY and the mystery strain you identified in Problem Set 3. Each segregant has been genotyped at marker positions across the genome (encoded as −1 for BY alleles and +1 for the mystery strain's alleles) and phenotyped for growth in YPD + chemical X.

Your task is to scan the genome for quantitative trait loci (QTLs) — regions where the inheritance of one parental allele vs. the other is correlated with the phenotype.

Part A: Genome-Wide LOD Scan

For each marker, calculate the Pearson correlation (r) between genotype and phenotype across all segregants. Convert to a LOD score using:

$$\text{LOD} = \frac{-n \ln(1 - r^2)}{2 \ln(10)} \tag{8.1}$$

where n is the number of segregants and r is the Pearson correlation coefficient. The LOD score is a $\log_{10}$ likelihood ratio: a LOD of 2 means the observed linkage is 100× more likely than expected by chance; a LOD of 3 means 1000×.

- Calculate the LOD score at every marker position across the genome.
- Plot the LOD score across genomic position (a LOD scan, analogous to a Manhattan plot). Color by chromosome.
- Which marker has the highest LOD score?

Part B: Permutation Threshold for Significance

A LOD of 2 or 3 as a fixed threshold does not account for the number of markers tested or the correlation structure among nearby markers. Use a permutation procedure to establish an empirical genome-wide significance threshold.

- Permute the assignment of phenotypes to genotypes 1000 times.
- For each permutation, calculate the LOD at every marker and record the maximum LOD across the genome.
- This gives you a null distribution of 1000 genome-wide maximum LOD scores.
- Compare the observed maximum LOD to this null distribution. Visualize the null distribution as a histogram and mark the observed maximum.
- Calculate a permutation-based p-value.
- Question: How could using a simple fixed LOD threshold (e.g., LOD > 2) lead to false positives or false negatives?

Part C: 1.5-LOD Drop Confidence Interval

The 1.5-LOD drop interval is a standard method for defining an approximate 95% confidence interval for the location of a causal variant underlying a QTL.

- Find the peak marker (highest LOD) for the largest QTL.
- Moving outward in both directions from the peak, find the markers whose LOD is ≥ 1.5 units below the peak LOD.
- The interval between these flanking markers is your 1.5-LOD drop confidence interval.
- Report the genomic coordinates (chromosome, start, end) of this interval.

Part D: Candidate Genes & Biological Hypothesis

- List the genes within the 1.5-LOD drop confidence interval. You can use SGD (Saccharomyces Genome Database) or a BED/GFF annotation file.
- Examine the functions of these genes.
- Given what you know from Problem Sets 1–3 about the strains, the experimental setup, and the QTL interval, form a hypothesis: what might chemical X be?
- Write a short paragraph explaining your reasoning.

You just followed the complete arc of a quantitative genetics experiment: phenotyping (growth curves), estimating heritability (mixed models), genotyping (short-read alignment), and genetic mapping (QTL scan with permutation-based significance and confidence intervals).
Each step required a different combination of Unix, R, statistics, and critical thinking. That integration is the point.

Appendix A: Tools & Installation

Every non-default tool referenced in this curriculum, with install commands and official documentation. Most bioinformatics software is installed into a conda environment (see Block 2, Sprint 6); a handful of core tools sit outside conda and should be installed first.

Core infrastructure (install once, before anything else)

- **Miniforge** — conda + mamba pre-configured for the conda-forge channel. Download and run the installer for your OS/architecture from https://github.com/conda-forge/miniforge.
- **Git** — macOS: `xcode-select -install` also installs Git as part of the Xcode Command Line Tools. Linux: `sudo apt install git` (Debian/Ubuntu) or `sudo dnf install git-all` (Fedora/RHEL). Docs: https://git-scm.com.
- **VS Code** — .dmg (macOS) or .deb / .rpm / tar.gz (Linux) installers at https://code.visualstudio.com.
- **Docker** — macOS: Docker Desktop, https://docs.docker.com/desktop/install/mac-install/. Linux: the per-distribution Engine install at https://docs.docker.com/engine/install/. Rootless alternative: Podman, https://podman.io.
- **Apptainer** (formerly Singularity) — HPC clusters usually provide a module (`module avail apptainer`). For a personal Linux install, see https://apptainer.org/docs/user/main/quick_start.html. Not supported natively on macOS.
- **R** — install the pre-built binary matching your OS/architecture from CRAN: https://cran.r-project.org.
- **RStudio Desktop** (recommended IDE for R) — install after R, free desktop edition: https://posit.co/download/rstudio-desktop/.
- **Homebrew** (macOS package manager, useful for everything in the next section) — one-line install at https://brew.sh.

Command-line utilities not always installed by default

- `tree` (directory visualization) — macOS: `brew install tree`. Debian/Ubuntu: `sudo apt install tree`. Fedora/RHEL: `sudo dnf install tree`.
- `tmux` (persistent terminal sessions, critical on HPC) — macOS: `brew install tmux`. Linux: `sudo apt install tmux` or `sudo dnf install tmux`. Docs: https://github.com/tmux/tmux/wiki.
- `jq` (JSON filtering on the command line) — macOS: `brew install jq`. Linux: `sudo apt install jq`. Docs: https://jqlang.github.io/jq/.
- `bc` (arbitrary-precision calculator; used in Block 1 Sprint 4) — usually default on macOS. On minimal Linux images: `sudo apt install bc`.
- `parallel` (GNU parallel, for command-line job parallelism) — macOS: `brew install parallel`. Linux: `sudo apt install parallel`.
- `curl` / `wget` — both default on macOS and most Linux distros; `sudo apt install curl wget` if either is missing.

Bioinformatics tools (install into a conda env)

All of these are available on the `bioconda` channel. Make one environment per project and install what you need:

```
mamba create -n biotools -c conda-forge -c bioconda \
  samtools bcftools bedtools fastp bwa-mem2 star \
  sra-tools plink2 regenie snakemake nextflow
mamba activate biotools
```

Channel order matters: `conda-forge` before `bioconda`. Channel-priority conflicts are the number-one source of conda installation failures. If you hit one, check your `.condarc`.

samtools	View/sort/index/filter BAM & CRAM	`htslib.org`
bcftools	Filter/merge/annotate VCFs	`htslib.org`
bedtools	Intersect, merge, closest genomic regions	`bedtools.readthedocs.io`
GATK	Variant calling (HaplotypeCaller, JointGenotyping)	`gatk.broadinstitute.org`
BWA-MEM2	Short-read DNA alignment	`github.com/bwa-mem2/bwa-mem2`
STAR	Splice-aware RNA-seq alignment	`github.com/alexdobin/STAR`
fastp	FASTQ QC + adapter trimming	`github.com/OpenGene/fastp`
SRA Toolkit (`fastq-dump`, `fasterq-dump`)	Download FASTQ from SRA	`github.com/ncbi/sra-tools`
PLINK 2.0	Genotype QC, GWAS, PCA	`cog-genomics.org/plink/2.0`
REGENIE	Mixed-model GWAS	`rgcgithub.github.io/regenie`
LDSC	Heritability + genetic correlation from summary stats	`github.com/bulik/ldsc`
SuSiE (R package)	Fine-mapping credible sets	`stephenslab.github.io/susieR`
CellRanger	10x Genomics read processing (registration required; not on bioconda)	`10xgenomics.com/support/software/cell-ranger`
Snakemake	Reproducible workflow DAGs	`snakemake.readthedocs.io`
Nextflow	Alternative workflow engine (nf-core pipelines)	`nextflow.io`

R and Python packages

R (CRAN). Inside an R session: `install.packages("tidyverse")` — one command pulls in `dplyr`, `ggplot2`, `readr`, `purrr`, `tidyr`, `stringr`, `forcats`. Also useful: `install.packages(c("data.table", "lme4", "broom"))`.

R (Bioconductor). Inside an R session:

```
install.packages("BiocManager")
BiocManager::install(c("GenomicRanges", "DESeq2",
                       "edgeR", "limma",
                       "SummarizedExperiment",
                       "rtracklayer", "GEOquery",
                       "recount3", "Seurat"))
```

Bioconductor home: https://bioconductor.org.

Python. Install into an active conda env:

```
mamba install -c conda-forge -c bioconda \
  pandas numpy matplotlib seaborn \
  scipy statsmodels scikit-learn \
  pysam biopython scanpy scikit-allel \
  umap-learn
```

The `pip` equivalent works too; prefer conda when a package wraps a C library (`pysam`, `scanpy`) so the pre-built binary is used.

Appendix B: Programming Concepts Cheat Sheet

The concepts you'll use over and over. Bookmark this page.

Dictionaries / hash maps	$O(1)$ lookup — the fastest way to find gene annotations, sample metadata
Vectorized operations	NumPy/R replaces slow loops — essential for million-variant math
Sparse matrices	scRNA-seq is 95% zeros — sparse storage cuts memory 10–100×
Generators	Stream through billion-line files without loading into RAM
Regular expressions	Parse biological IDs, extract annotations, validate formats
Matrix algebra	PCA, linear models, kinship matrices — all matrix operations
Parallel processing	Chromosome-parallel GWAS, multi-sample processing
Error handling	Graceful recovery from malformed records in messy real data
Unit testing	Verify your functions work before running on expensive real data
Documentation	Future-you is your most important collaborator

Appendix C: Reading List (Prioritized)

Start Here (the essentials)

- StatQuest YouTube (Josh Starmer) — watch the PCA, DESeq2, and p-value episodes first
- Software Carpentry: Unix Shell workshop (free, interactive, excellent)
- Scanpy PBMC3k tutorial (the canonical single-cell starting point)
- Marees et al. – A tutorial on conducting GWAS (step-by-step)
- R for Data Science (r4ds.hadley.nz) — free, skip to Ch. 3 if you know basics

Go Deeper (when you're ready)

- Buffalo – *Bioinformatics Data Skills* (the Unix + workflows bible)
- McKinney – *Python for Data Analysis*, 3rd Ed (the Pandas bible)
- Holmes & Huber – *Modern Statistics for Modern Biology* (free online)
- Love, Huber, Anders – DESeq2 paper (Genome Biology, 2014)
- Heumos et al. – Best practices for single-cell analysis (Nat Rev Genet, 2023)
- Uffelmann et al. – Genome-wide association studies (Nat Rev Methods Primers, 2021)
- Bulik-Sullivan et al. – LD Score regression (Nature Genetics, 2015)

Reference (keep on your shelf)

- Gillespie – *Population Genetics: A Concise Guide*
- Efron – *Large-Scale Inference*
- Storey & Tibshirani – FDR paper (PNAS, 2003)
- Wang et al. – SuSiE (JRSS-B, 2020)
- Wilson et al. – Good enough practices in scientific computing (PLOS Comp Bio, 2017)

Online Tools & Databases

- explainshell.com — paste any Unix command to see what each flag does
- Bioconductor workflows — bioconductor.org/packages/release/workflows/
- GWAS Catalog — ebi.ac.uk/gwas/
- PGS Catalog — pgscatalog.org
- nf-core — nf-co.re (community Nextflow pipelines)
- Anthropic Prompt Engineering Guide — docs.anthropic.com

12-Block Overview

0	Setup	Install everything	Git repo with journal.md
1	Unix	Filesystem, pipes, text tools	One-liner pipeline for VCF analysis
2	Unix	Bash, conda/mamba, containers, Git	Automated QC script on GitHub
3	Python	Data structures, parsing, idiomatic Python	FASTA parser with tests
4	Python	Pandas, plotting, bio libs	Manhattan + QQ plots from GWAS data
5	R	Lists, apply family, tidyverse, Bioconductor	GRanges overlaps + data loaded from GEO
6	R	DESeq2, ggplot2	DE analysis + Problem Set 1 (Growth Curves)
7	Stats	Distributions, testing, models	Simulated GWAS with FDR analysis
8	Stats	PCA, UMAP, Bayes concepts	Problem Set 2 (Heritability & Mixed Models)
9	Apps	scRNA-seq full pipeline	Annotated cell clusters from real data
10	Apps	GWAS pipeline, post-GWAS	Problem Set 3 (Short-Read Alignment & Strain ID)
11	Meta	AI prompting, right questions	AI research journal
12	Capstone	Pipeline + project	Problem Set 4 (QTL Mapping) or custom capstone